

Supervised Learning

Linear Regression and Loss Functions

Firuz Kamalov

MAI 604: Deep Learning

January 16, 2026

Contents

- ① Course Information
- ② Artificial Intelligence (AI)
- ③ Machine Learning (ML) Algorithms
- ④ Supervised Learning

Contents

- ① Course Information
- ② Artificial Intelligence (AI)
- ③ Machine Learning (ML) Algorithms
- ④ Supervised Learning

Required text:

Prince, S. J.D. ,

Understanding Deep Learning, MIT Press, 2023

ISBN: 9780262048644

HTML version of the book is available at

<https://udlbook.github.io/udlbook/>

Required text:

Zhang, A. et al.,

Dive into Deep Learning,

Cambridge University Press, 2023

ISBN: 978-1009389433

HTML version of the book is available at

<https://d2l.ai>

Prerequisites

Multivariate Calculus:

- Derivatives
- Chain Rule
- Partial derivatives
- Multivariate Chain Rule
- Gradient etc.

Prerequisites

Probability and Statistics:

- Random variables
- Probability distributions
- Expectations etc.

Prerequisites

Python:

- Data types
- For loops
- If...elif...else
- Functions
- Classes/Objects etc.

Grading

Assignments, Quizzes, Exams

$$\begin{aligned}\text{Grade} = & 0.15 \cdot \text{Weekly assignments} \\ & + 0.25 \cdot \text{Midterm exam (week 8)} \\ & + 0.35 \cdot \text{Final exam (week 16)} \\ & + 0.25 \cdot \text{Group project}\end{aligned}$$

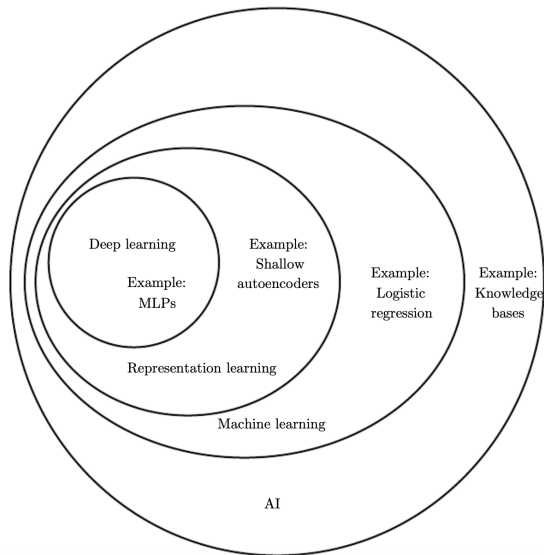
Homework Assignments

- Except when especially noted, assignments will be due each Friday at 11:59 pm.
- Solutions to the assignments submitted later than 1, 2, and 3 days after the due date will be penalized by 10%, 50%, and 100%, respectively.
- Complete code (.ipynb files) needs to be submitted on MyCUD in a form of zip file.

Contents

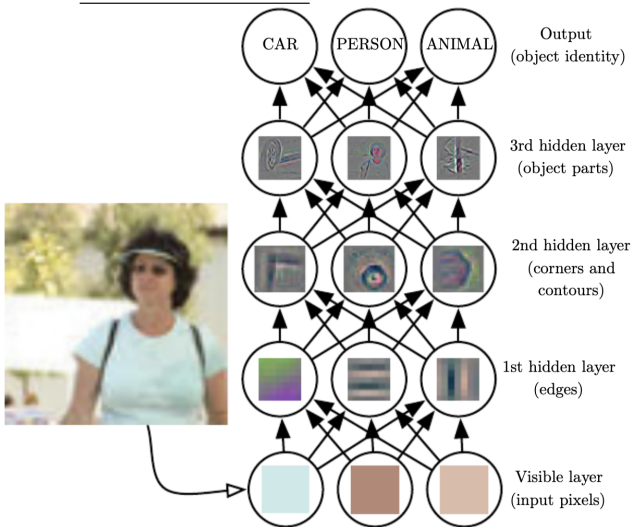
- ① Course Information
- ② Artificial Intelligence (AI)
- ③ Machine Learning (ML) Algorithms
- ④ Supervised Learning

Artificial Intelligence: ML and DL



Example of DL: Convolutional Neural Network (CNN)

Deep Learning as multistage learning of data representations:



Contents

- ① Course Information
- ② Artificial Intelligence (AI)
- ③ Machine Learning (ML) Algorithms
- ④ Supervised Learning

- Linear Regression
- Logistic Regression
- Linear Discriminant Analysis (LDA)
- Quadratic Discriminant Analysis (QDA)
- K Nearest Neighbors (KNN)
- Random Tree / Forest
- Support Vector Machine (SVM)
- Artificial Neural Networks (NN)

- K-means Clustering
- Hierarchical Clustering
- Mixture Model
- Expectation–maximization (EM) Algorithm
- Method of Moments
- Artificial Neural Networks (NN)

Part I: Fundamentals

- **Supervised learning**
- Shallow neural networks
- Deep neural networks
- Loss functions
- Fitting models
- Gradients and initialization
- Measuring performance
- Regularization

Part II: Architectures & Advanced

- Convolutional networks
- Residual networks
- Recurrent networks
- Embeddings
- Transformers and Large Language Models
- Variational autoencoders
- Generative adversarial networks

Contents

- ① Course Information
- ② Artificial Intelligence (AI)
- ③ Machine Learning (ML) Algorithms
- ④ Supervised Learning

Supervised Learning Overview

Definition

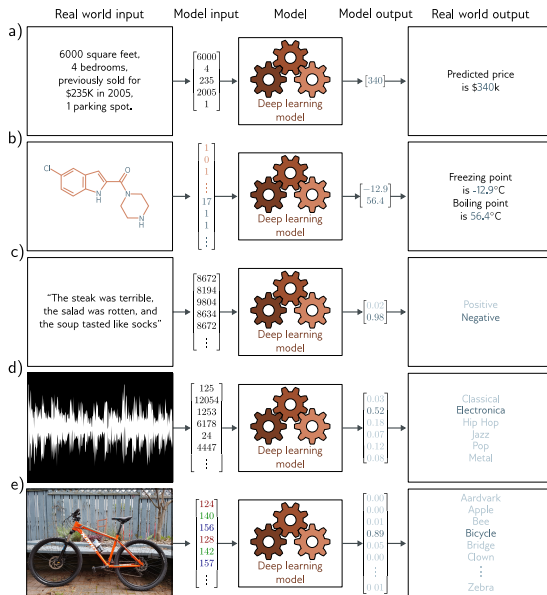
A **supervised learning model** defines a mapping from one or more inputs ($\mathbf{x}$) to one or more outputs ($\mathbf{y}$).

Core Terminology:

- **Inference:** Computing the output $\mathbf{y}$ from input $\mathbf{x}$ using the model equation.
- **Parameters (ϕ):** Adjustable values within the model that determine the specific relationship.
- **Training (Learning):** The process of finding the best parameters ϕ using a dataset of input/output pairs $\{\mathbf{x}_i, \mathbf{y}_i\}$.

Goal: Predict outputs for new inputs where the true output is unknown.

Supervised Learning Tasks



1D Linear Regression Model

We start with a simple model that describes the relationship between a single input x and output y as a straight line.

$$y = f[x, \phi] = \phi_0 + \phi_1 x \quad (1)$$

Parameters:

- ϕ_0 : The y-intercept.
- ϕ_1 : The slope.

Different choices of ϕ create a "family" of possible relationships (lines).

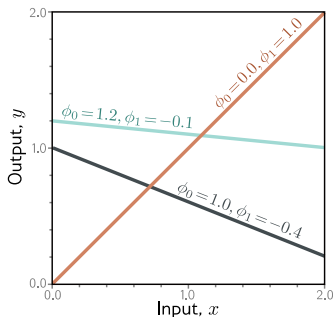


Figure: A family of functions defined by Eq. 1

The Loss Function

To train the model, we need to quantify how well a specific set of parameters ϕ fits the training data. We use a **Loss Function** (L).

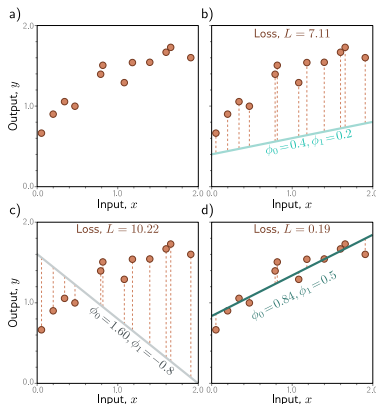
Least Squares Loss: We calculate the difference between the model prediction $f[x_i, \phi]$ and the true output y_i , square it, and sum over all I data points:

$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$

$$L[\phi] = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

Visualizing the Loss

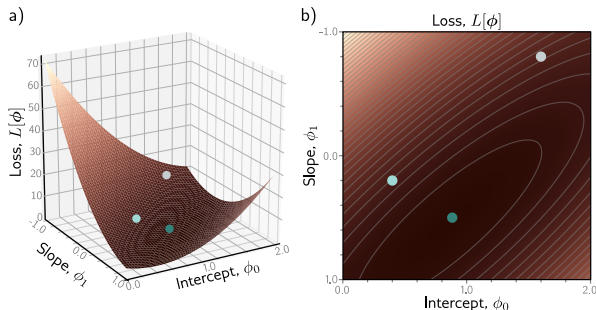
The loss corresponds to the sum of the squared lengths of the vertical residuals (orange dashed lines).



(b) Poor fit ($L = 7.11$). (c) Worse fit ($L = 10.22$). (d) Good fit ($L = 0.19$).

The Loss Surface

Since our model has two parameters (ϕ_0, ϕ_1), the loss function $L[\phi]$ forms a surface. We want to find the lowest point on this surface.



Left: 3D Surface. Right: Heatmap (Contour plot). Brighter regions indicate higher loss; darker regions indicate lower loss.

Training: Finding the Minimum

Training aims to find the parameters $\hat{\phi}$ that minimize the loss:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L[\phi]]$$

Gradient Descent Strategy:

- 1 Initialize ϕ randomly (Position 0).
- 2 Measure the slope (gradient) of the loss surface.
- 3 Step downhill (opposite to gradient).
- 4 Repeat until the bottom is reached (Position 4).

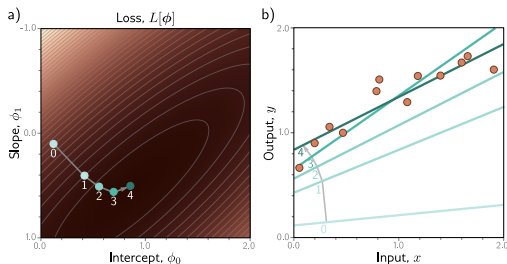


Figure: Iterative training process.

Testing and Generalization

After training, we evaluate the model on **Test Data** (data not seen during training).

- **Generalization:** The ability of the model to predict accurately on new inputs.
- **Underfitting:** The model is too simple to capture the underlying relationship (e.g., trying to fit a line to a complex curve).
- **Overfitting:** The model is too expressive. It memorizes the noise in the training data rather than the true pattern, leading to poor predictions on test data.

Python Implementation: 1D Linear Regression

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import mean_squared_error
5
6 # 1. Prepare Data (Approx. from Figure 2.2)
7 x_data = np.array([0.03, 0.19, 0.34, 0.46, 0.78, 0.81,
8                    1.08, 1.18, 1.39, 1.60, 1.65, 1.90])
9 y_data = np.array([0.67, 0.85, 1.05, 1.00, 1.40, 1.50,
10                   1.30, 1.54, 1.55, 1.69, 1.80, 1.60])
11 X = x_data.reshape(-1, 1) # Reshape to 2D array
12
13 # 2. Split Data: Reserve 33% for testing (Generalization check)
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y_data, test_size=0.33, random_state=42
16 )
17
18 # 3. Train Model (Fit only on training data)
19 model = LinearRegression()
20 model.fit(X_train, y_train)
21
22 # 4. Test Model (Predict on unseen test data)
23 y_pred = model.predict(X_test)
24
25 # 5. Evaluate Performance (Calculate Loss)
26 test_loss = mean_squared_error(y_test, y_pred)
```

Summary

- ① **Model:** $y = f[\mathbf{x}, \phi]$ maps inputs to outputs.
- ② **Loss Function:** $L[\phi]$ quantifies the mismatch between predictions and data.
- ③ **Training:** We optimize parameters ϕ to minimize $L[\phi]$.
- ④ **Inference:** We use the trained model to make predictions.

Thank you. Questions?